Task 1

Develop a C program using fork() that creates a parent and child process . Display the process ID(PID), Parent Process ID(PPID), and process states at different stages of execution

Source Code:

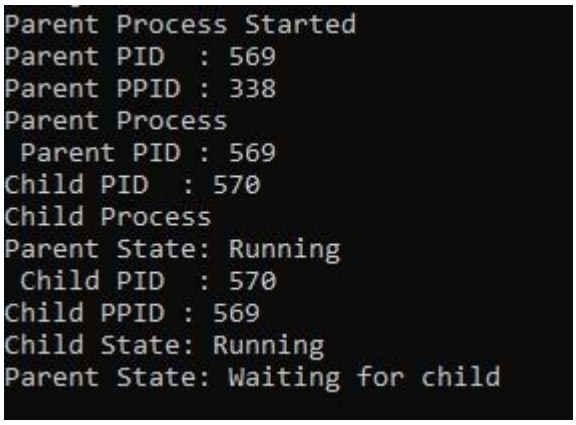
```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/wait.h>
#include<stdlib.h>

Int main(){
pid_t pid;
printf("Parent Process Started\n");
printf("Parent PID : %d\n", getpid());
printf("Parent PPId :%d\n", getppid());
pid=fork();
if(pid<0){
printf("Fork failed\n");
return 1;
} else if(pid == 0) {
printf("Child Process\n");
printf("Child PID : %d\n",getppid());
printf("Child State: Running\n");
sleep(5);
printf("Child State: Terminating\n");
exit(0);
}
else{
printf("Parent Process\n"); printf("Parent PID :
```

```
%d\n", getpid());        printf("Child PID  : %d\n",
pid);        printf("Parent State: Running\n");
sleep(2);
      printf("Parent State: Waiting for child\n");        wait(NULL);


      printf("Parent State: Child Terminated\n");
   }    return 0;
}
```

Output:



Task 2

Design an experiment to observe process state transitions (Ready, Running , Waiting , Terminated) using Linux monitoring tools such as ps , top, and /proc. Document the observations.

Source Code:

```
#include<stdio.h>
#include <unistd.h>
#include<sys/wait.h>
Int main(){    pid_t
Pid=fork();   if(pid
==0){
printf("Child process started. PID = %d\n", getpid());
sleep(10);
```

```
printf("Child process terminated.\n");
   }    else {
      printf("Parent process started. PID = %d\n", getpid());
printf("Child PID = %d\n", pid);        wait(NULL);
      printf("Parent: Child terminated.\n");
   }    return 0;
}
```

```
Parent process started. PID = 1036
Child process Started. PID= 1037
Child PID = 1037
Child process terminated.
Parent: Child terminated.
```

1.ps: displays the processes that are currently running in the system.

```
root@DESKTOP-VMN932K:~# ps
    PID TTY          TIME CMD
    338 pts/0    00:00:00 bash
   1061 pts/0    00:00:00 ps
```

2.top:It is used to monitor processes. It displayed the PID, user , cpu details, memory usage, and process state.

```
root@DESKTOP-VMN932K:~# top
top - 03:36:31 up 35 min,  1 user,  load average: 0.04, 0.01, 0.00
Tasks:  25 total,   1 running,  24 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.2 sy,  0.0 ni, 99.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7880.6 total,   7420.4 free,    453.6 used,    156.1 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7427.0 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   21844  13148   9796 S   0.0   0.2   0:01.11 systemd
      2 root      20   0    3180   2204   2072 S   0.0   0.0   0:00.00 init-systemd(Ub
      9 root      20   0    3196   2124   2008 S   0.0   0.0   0:00.00 init
     58 root      19  -1   34056  12672  11600 S   0.0   0.2   0:00.41 systemd-journal
    108 root      20   0   25284   6632   5172 S   0.0   0.1   0:00.36 systemd-udevd
    114 systemd+  20   0   21344  13180  11056 S   0.0   0.2   0:00.15 systemd-resolve
    115 systemd+  20   0   91036   7968   6996 S   0.0   0.1   0:00.16 systemd-timesyn
    159 root      20   0    4244   2684   2432 S   0.0   0.0   0:00.01 cron
```

3. /proc: It is a directory

```
root@DESKTOP-VMN932K:~# /proc
-bash: /proc: Is a directory
```